RAG-Asistant — Smart HelpDesk Assistant

RAG-Asistant is an AI-powered helpdesk assistant that indexes knowledge base articles using hybrid search (BM25 + Pinecone), retrieves relevant information via LlamaIndex, and generates accurate, streaming responses with source citations — all in a modern Streamlit interface.

Prerequisites

- Python 3.12+
- pip
- Pinecone API Key
- Gemini API Key

Setup

1. Clone the Repository

```
### 2. Create Virtual Environment
**Windows:**
```bash
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Mac/Linux:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

3. Configure Environment

Create a `.env` file at the root:

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_LLM_MODEL=gemini-2.5-flash
GEMINI_EMBEDDING_MODEL=gemini-embedding-2-preview
PINECONE_API_KEY=your_pinecone_api_key_here
PINECONE_INDEX_NAME=askly-index
PINECONE_ENVIRONMENT=us-east-1
PINECONE_EMBEDDING_MODEL=llama-text-embed-v2
```
```

```
PINECONE_EMBEDDING_ENABLED=true
EMBEDDING_DIM=1024
CHUNK_SIZE=512
CHUNK_OVERLAP=50
TOP_K_RESULTS=10
BM25_TOP_K=10
RAGAS_ENABLED=true
RAGAS_GEMINI_ENABLED=false
```

Environment Variables:

- `GEMINI_API_KEY`: Google Gemini API key for LLM and embedding
- `GEMINI_LLM_MODEL`: Gemini model for generation (default: gemini-2.5-flash)
- `GEMINI_EMBEDDING_MODEL`: Gemini model for embeddings (default: gemini-embedding-2-preview)
- `PINECONE_API_KEY`: Pinecone API key for vector database
- `PINECONE_INDEX_NAME`: Name of your Pinecone index (default: askly-index)
- `PINECONE_ENVIRONMENT`: Pinecone region (default: us-east-1)
- `PINECONE_EMBEDDING_MODEL`: Pinecone embedding model (default: llama-text-embed-v2)
- `PINECONE_EMBEDDING_ENABLED`: Use Pinecone for embeddings instead of Gemini (default: true)
- `EMBEDDING_DIM`: Embedding dimension (default: 1024)
- `CHUNK_SIZE`: Document chunk size in words (default: 512)
- `CHUNK_OVERLAP`: Chunk overlap in words (default: 50)
- `TOP_K_RESULTS`: Number of Pinecone results to retrieve (default: 10)
- `BM25_TOP_K`: Number of BM25 results to retrieve (default: 10)
- `RAGAS_ENABLED`: Enable RAGAS quality evaluation (default: true)
- `RAGAS_GEMINI_ENABLED`: Use Gemini for RAGAS evaluation (default: false - uses heuristic only)

🗺️ Feature Walkthrough

🩹 Feature 1 — Document Ingestion

In the **Sources** panel:

1. Upload PDF or Markdown files (e.g., Company Policies, Onboarding Guides).
2. Askly's **Ingestion Pipeline** will:
    - Parse text with PyMuPDF.
    - Chunk content into 512-word sliding windows.
    - Embed chunks using Pinecone or Gemini embedder (configurable).
    - Upsert vectors into Pinecone and update the local BM25 index.

🔍 Feature 2 — Intelligent Hybrid Search

In the **Ask** panel:

- Ask a question like: "*What is our hybrid work policy?*"
- **Hybrid Search** triggers:

- **Dense:** Pulls semantic matches from Pinecone.
- **Sparse:** Pulls keyword matches from BM25.
- **Fusion:** Orchestrated by **LlamaIndex QueryFusionRetriever** using **RRF (Reciprocal Rank Fusion)**.

⚡ Feature 3 — Streaming Context-Grounded Generation

Askly streams answers in real-time using a LlamaIndex query engine backed by Gemini:

- **Real-time tokens:** Answers appear word-by-word (~200ms first token).
- **Source Citations:** Every answer displays source tiles showing document name, rank, relevance score, and page number.
- **Follow-up Suggestions:** AI generates 2-3 clickable follow-up question chips.
- **Conversational Fallback:** When no relevant documents are found (e.g., greetings like "hi"), the system uses the LLM to generate natural responses instead of returning empty results.
- **LLM Failure Fallback:** When the LLM fails (quota, network, or API errors), the system displays retrieved document chunks with error details instead of a complete failure.

👍 Feature 4 — Answer Feedback System

Every AI answer includes inline feedback buttons:

- 👍 **/** 👎 **Buttons** — rate answer quality with one click.
- **Feedback persisted** to `data/feedback.json` for analytics.
- **Satisfaction rate** computed and displayed in the Analytics dashboard.

📋 Feature 5 — Export & Share

Share knowledge across your team:

- 📋 **Copy to clipboard** — one-click copy of any answer.
- 📥 **Export conversation** — download the full chat as a Markdown file.

🔬 Feature 6 — Document Explorer

A dedicated **Explorer** page to inspect indexed chunks:

- **Search** across all chunks by keyword.
- **Filter** by specific document.
- **Browse** with paginated chunk cards showing doc name, page, ID, and content.
- Useful for debugging "why didn't it find my answer?"

📊 Feature 7 — Intelligence Dashboard

Click **Analytics** to view comprehensive performance metrics:

- **5 KPI Cards:** Total Queries, Faithfulness, Relevance, Precision, Satisfaction Rate.
- **Score Trend Charts:** Line chart showing RAGAS metrics over time.
- **Per-Query Bar Chart:** Overall score for each query.
- **Feedback Distribution:** Visual bar showing 👍 vs 👎 percentages.
- **Knowledge Gaps:** Automatically identifies queries with low confidence scores.
- **Recent Query History:** With color-coded quality scores.

Evaluation uses RAGAS library with Langchain wrappers for Gemini fallback when needed.

💬 Feature 8 — Multi-Turn Conversations

- Full conversation history maintained across turns.
- Context sent to Gemini for follow-up question understanding.
- Sessions persisted to disk — survive app restarts.

🗂️ Feature 9 — Conversation History

A dedicated **Recents** page to manage conversation history:

- **Browse** all past conversations with timestamps.
- **Switch** between conversations to continue previous discussions.
- **Delete** conversations to clean up history.
- Conversations persisted as JSON files in `data/sessions/`.

🗀 Project Structure

▶ Click to expand

```
askly/
├── streamlit/
│   └── app.py                      # Main Entry Point (Geist UI + Controller)
├── askly/
│   ├── core/
│   │   ├── retrieval/              # Hybrid Search (BM25 + Pinecone)
│   │   │   ├── bm25_store.py       # BM25 lexical search
│   │   │   ├── pinecone_store.py   # Pinecone vector database
│   │   │   └── hybrid.py           # RRF fusion orchestration
│   │   ├── ingestion/              # Text Parsing, Chunking & Indexing
│   │   │   ├── gemini_embedder.py  # Gemini embedding model
│   │   │   ├── pinecone_embedder.py# Pinecone embedding model
│   │   │   ├── chunker.py          # Document chunking
│   │   │   ├── parser.py           # PDF/Markdown parsing
│   │   │   └── pipeline.py         # Ingestion orchestration
│   │   ├── generation/             # RAG Prompt Engineering + Streaming
(Gemini)
│   │   │   └── generator.py        # Streaming response generator
│   │   ├── evaluation/             # RAGAS metrics & Hallucination detection
```

```
│   │   │   └── ragas_eval.py       # Quality evaluation
│   │   ├── conversation/           # Multi-turn session persistence
│   │   │   └── session.py          # Chat session management
│   │   └── feedback/               # User feedback (👍/👎) persistence
│   │       └── manager.py          # Feedback storage
│   ├── config.py                   # Pydantic environment configuration
│   ├── models/
│   │   └── schemas.py              # Data models for Chunks & Evaluations
│   └── utils/
│       └── logger.py               # Structured system logging
├── data/                           # Local storage (BM25 index, Sessions,
Feedback)
├── logs/                           # Application logs
├── requirements.txt                # Project Dependencies
├── .env.example                    # Environment variables template
└── ARCHITECTURE.md                 # System Architecture Deep-Dive
```

🛠️ Technology Stack

Component	Technology
LLM	Google Gemini (Streaming)
Vector DB	Pinecone (Serverless)
Embeddings	Pinecone (default) or Gemini (configurable)
Orchestration	LlamaIndex QueryFusionRetriever + RetrieverQueryEngine
Keyword Search	rank_bm25
Score Fusion	Reciprocal Rank Fusion
UI Framework	Streamlit
Document Parsing	PyMuPDF + python-markdown
RAG Evaluation	RAGAS + Langchain (for Gemini wrapper fallback)
Data Validation	Pydantic
Data Analytics	Pandas
Backend	Python 3.12+